Artificial Neural Network (ANN) (3)

Dr. Mohammed Brahimi and Dr. Aicha Boutorh

1. Extensions and Variations
   - Vanishing Gradients and ReLUs
   - Weight Initialization and Unstable Gradients
   - Handling Categorical Target Features: Softmax Output Layers and Cross-Entropy Loss Functions
   - Early Stopping and Dropout: Preventing Overfitting

Extensions and Variations

Vanishing Gradients and ReLUs

$$
\begin{aligned}
\frac{\partial \mathcal{E}}{\partial w_{i,k}} &= \frac{\partial \mathcal{E}}{\partial a_i} \times \frac{\partial a_i}{\partial z_i} \times \frac{\partial z_i}{\partial w_{i,k}} \\
&= \qquad \boldsymbol{\delta}_i \qquad \times \frac{\partial z_i}{\partial w_{i,k}}
\end{aligned}
\tag{1}
$$

Vanishing Gradients and ReLUs

$$\text{i} \rightarrow \text{j} \rightarrow \text{k}$$

$$\begin{aligned}
\delta_i &= \frac{\partial \mathcal{E}}{\partial a_i} \times \frac{\partial a_i}{\partial z_i} \\
&= \overbrace{w_{j,i} \times \delta_j}^{\delta_i} \times \frac{\partial a_i}{\partial z_i} \\
&= w_{j,i} \times \overbrace{w_{k,j} \times \delta_k \times \frac{\partial a_j}{\partial z_j}}^{\delta_j} \times \frac{\partial a_i}{\partial z_i} \\
&= w_{j,i} \times w_{k,j} \times \overbrace{\frac{\partial \mathcal{E}}{\partial a_k} \times \frac{\partial a_k}{\partial z_k}}^{\delta_k} \times \frac{\partial a_j}{\partial z_j} \times \frac{\partial a_i}{\partial z_i}
\end{aligned} \tag{2}$$

Vanishing Gradients and ReLUs

$$rectifier(z) = max(0, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & otherwise \end{cases} \tag{3}$$

$$\frac{d}{dz} rectifier(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & otherwise \end{cases} \tag{4}$$

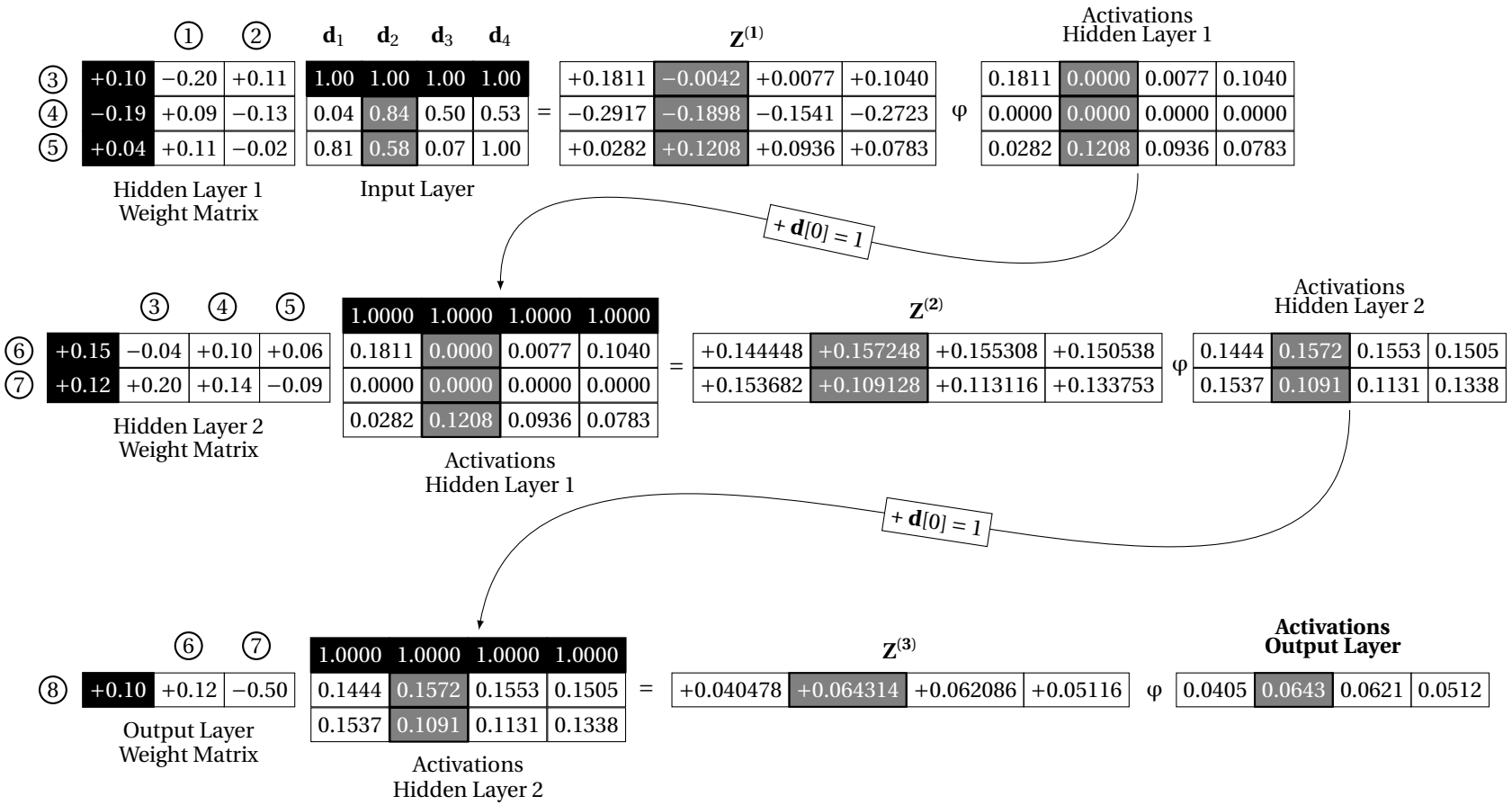


Figure 1: The forward pass of the examples listed in Table **??**[??] through the network in Figure **??**[??] when all the neurons are ReLUs.

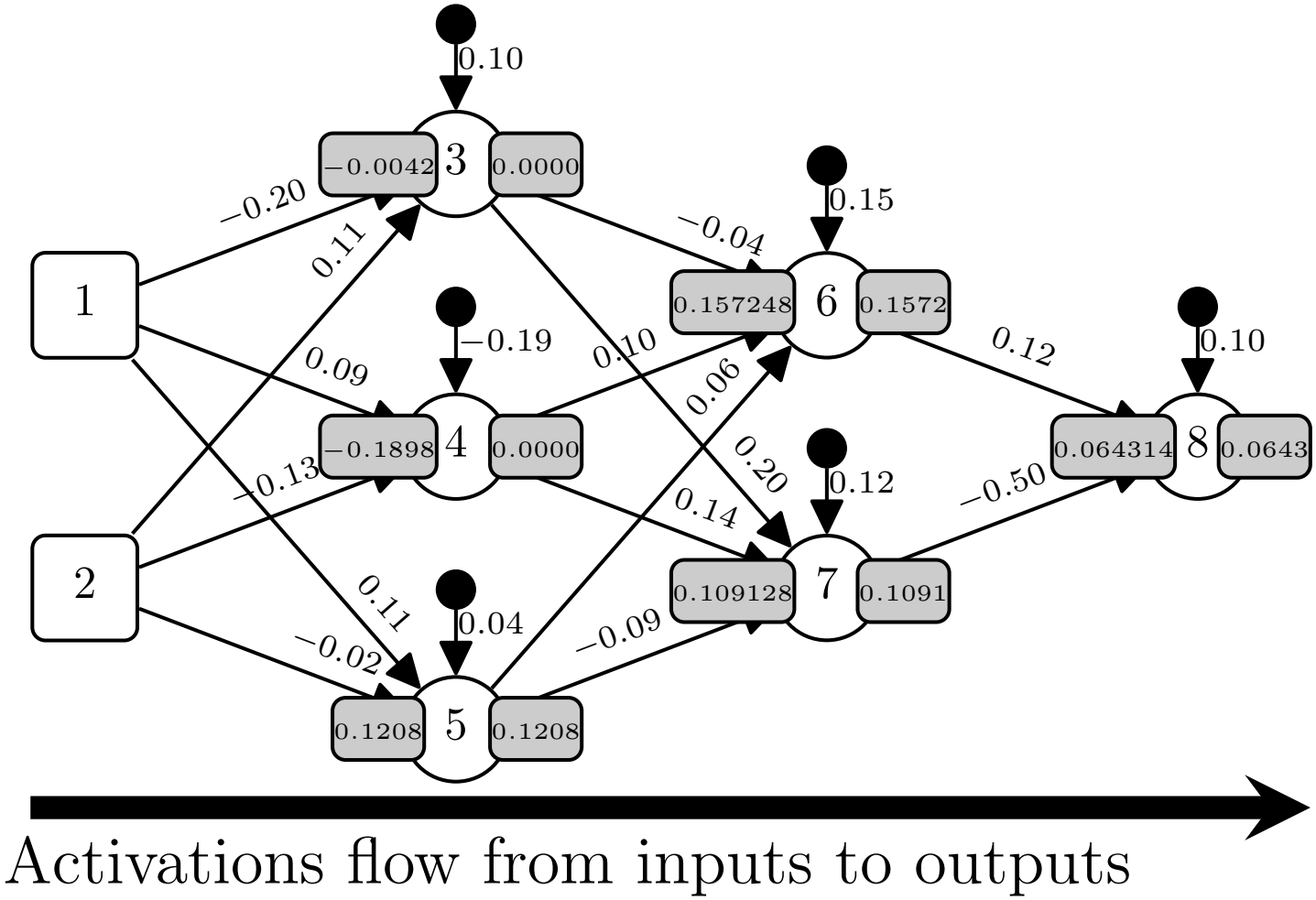


Figure 2: An illustration of the forward propagation of $\mathbf{d}_2$ through the ReLU network showing the weights on each connection, and the weighted sum z and activation a value for each neuron in the network.

Table 1: The per example error of the ReLU network after the forward pass illustrated in Figure 1[7], the per example $\partial\mathcal{E}/\partial a_8$, and the **sum of squared errors** for the ReLU model.

	$\mathbf{d}_1$	$\mathbf{d}_2$	$\mathbf{d}_3$	$\mathbf{d}_4$
Target	0.9400	0.1300	0.5700	0.3600
Prediction	0.0405	0.0643	0.0621	0.0512
Error	0.8995	0.0657	0.5079	0.3088
$\partial\mathcal{E}/\partial a_8$: Error $\times -1$	-0.8995	-0.0657	-0.5079	-0.3088
Error^2	0.80910025	0.00431649	0.25796241	0.09535744
SSE:				0.58336829

Table 2: The $\partial a/\partial z$ for each neuron for $\mathbf{d}_2$ rounded to four decimal places.

NEURON	z	$\partial a/\partial z$
3	-0.004200	0
4	-0.189800	0
5	0.120800	1
6	0.157248	1
7	0.109128	1
8	0.064314	1

$$
\begin{aligned}
\delta_k &= \frac{\partial \mathcal{E}}{\partial a_k} \times \frac{\partial a_k}{\partial z_i} \\
\delta_8 &= -0.0657 \times 1.0 \\
&= -0.0657 \\
\delta_7 &= \left(\delta_8 \times w_{8,7}\right) \times \frac{\partial a_7}{\partial z_7} \\
&= (-0.0657 \times -0.50) \times 1 \\
&= 0.0329 \\
\delta_6 &= \left(\delta_8 \times w_{8,6}\right) \times \frac{\partial a_6}{\partial z_6} \\
&= (-0.0657 \times 0.12) \times 1 \\
&= -0.0079 \\
\delta_5 &= \left(\left(\delta_6 \times w_{6,5}\right) + \left(\delta_7 \times w_{7,5}\right)\right) \times \frac{\partial a_5}{\partial z_5} \\
&= ((-0.0079 \times 0.06) + (0.0329 \times -0.09)) \times 1 \\
&= -0.0034 \\
\delta_4 &= \left(\left(\delta_6 \times w_{6,4}\right) + \left(\delta_7 \times w_{7,4}\right)\right) \times \frac{\partial a_4}{\partial z_4} \\
&= ((-0.0079 \times 0.10) + (0.0329 \times 0.14)) \times 0 \\
&= 0 \\
\delta_3 &= \left(\left(\delta_6 \times w_{6,3}\right) + \left(\delta_7 \times w_{7,3}\right)\right) \times \frac{\partial a_3}{\partial z_3} \\
&= ((-0.0079 \times -0.04) + (0.0329 \times 0.20)) \times 0 \\
&= 0
\end{aligned}
\tag{5}
$$

Table 3: The ReLU network's per example prediction, error, and the sum of squared errors after training has converged to an $SSE < 0.0001$.

	$\mathbf{d}_1$	$\mathbf{d}_2$	$\mathbf{d}_3$	$\mathbf{d}_4$
Target	0.9400	0.1300	0.5700	0.3600
Prediction	0.9487	0.1328	0.5772	0.3679
Error	-0.0087	-0.0028	-0.0072	-0.0079
Error^2	0.00007569	0.00000784	0.00005184	0.00006241
SSE:				0.00009889

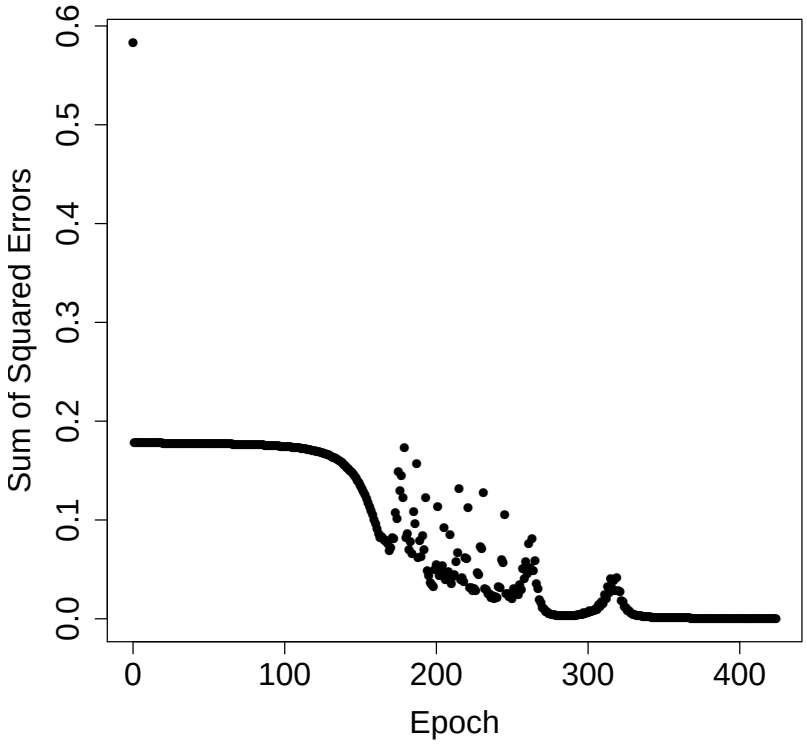


Figure 3: A plot showing how the sum of squared errors of the ReLU network changed during training when $\alpha = 0.2$.

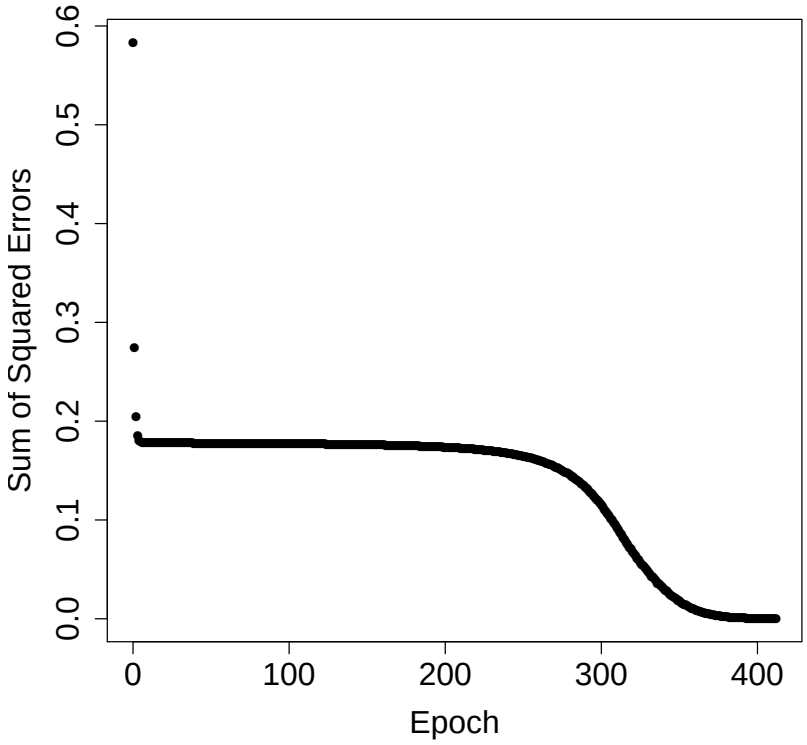


Figure 4: A plot showing how the sum of squared errors of the ReLU network changed during training when $\alpha = 0.1$.

$$rectifier_{leaky}(z) = \begin{cases} z & \text{if } z > 0 \\ 0.01 \times z & otherwise \end{cases} \tag{6}$$

$$\frac{d}{dz} rectifier_{leaky}(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0.01 & otherwise \end{cases} \tag{7}$$

$$rectifier_{parametric}(z_i) = \begin{cases} z_i & \text{if } z_i > 0 \\ \lambda_i \times z_i & otherwise \end{cases} \tag{8}$$

$$\frac{d}{dz} rectifier_{parametric}(z_i) = \begin{cases} 1 & \text{if } z_i > 0 \\ \lambda_i & otherwise \end{cases} \tag{9}$$

Vanishing Gradients and ReLUs

$$\frac{\partial \mathcal{E}}{\partial \lambda_i} = \frac{\partial \mathcal{E}}{\partial a_i} \times \frac{\partial a_i}{\partial \lambda_i} \tag{10}$$

$$\frac{\partial a_i}{\partial \lambda_i} = \begin{cases} 0 & \text{if } z_i > 0 \\ z_i & otherwise \end{cases} \tag{11}$$

$$\lambda_i \leftarrow \lambda_i - \alpha \times \frac{\partial \mathcal{E}}{\partial \lambda_i} \tag{12}$$

Weight Initialization and Unstable Gradients

$$\delta_i = \underbrace{w_{j,i} \times w_{k,j} \times}_{\substack{\text{extreme weights} \\ \rightarrow \\ \text{unstable gradients}}} \frac{\partial \mathcal{E}}{\partial a_k} \underbrace{\times \frac{\partial a_k}{\partial z_k} \times \frac{\partial a_j}{\partial z_j} \times \frac{\partial a_i}{\partial z_i}}_{\substack{\text{extreme weights} \\ \rightarrow \\ \text{saturated activations} \\ \rightarrow \\ \text{vanishing gradients}}} \tag{13}$$

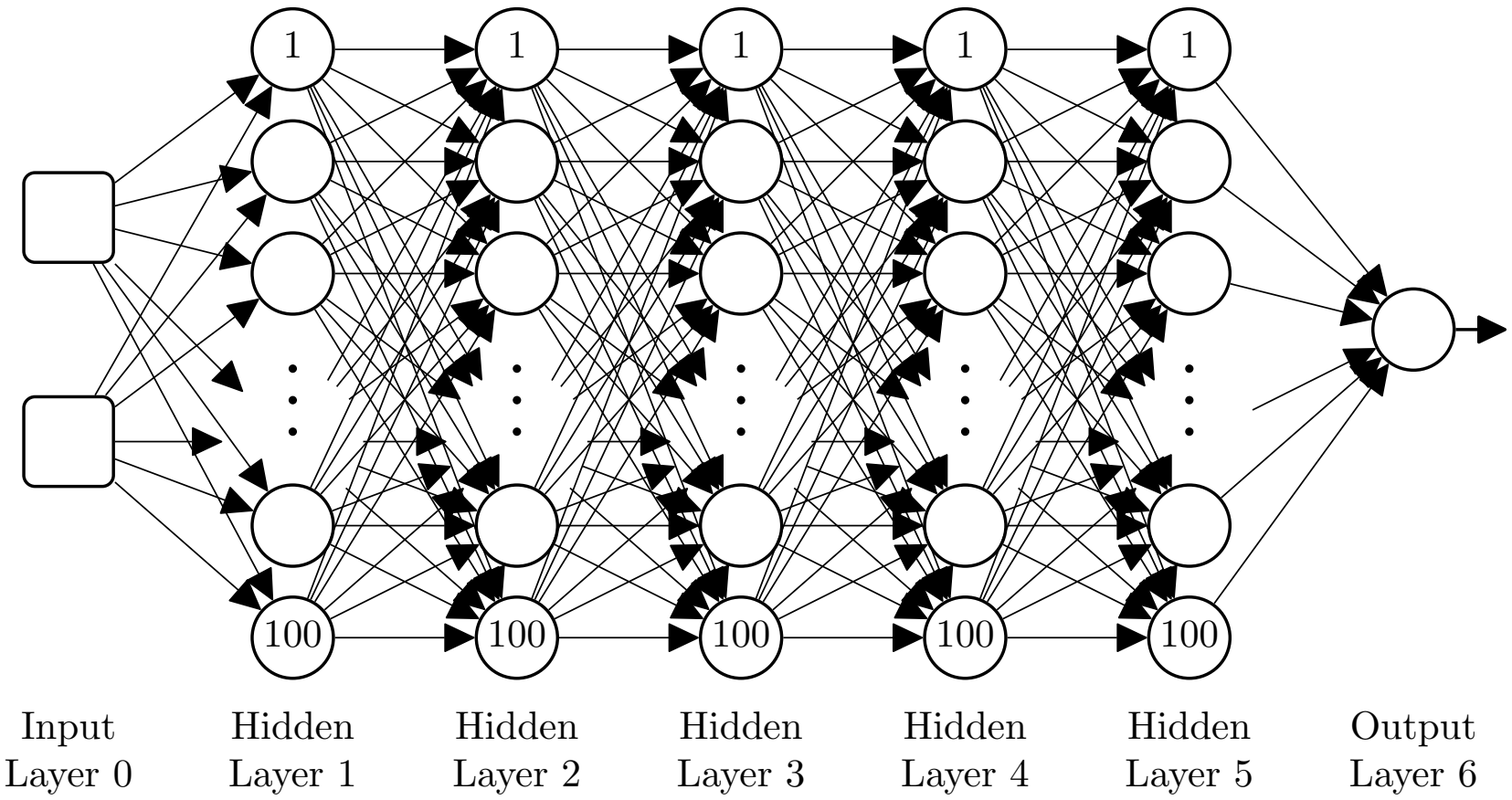
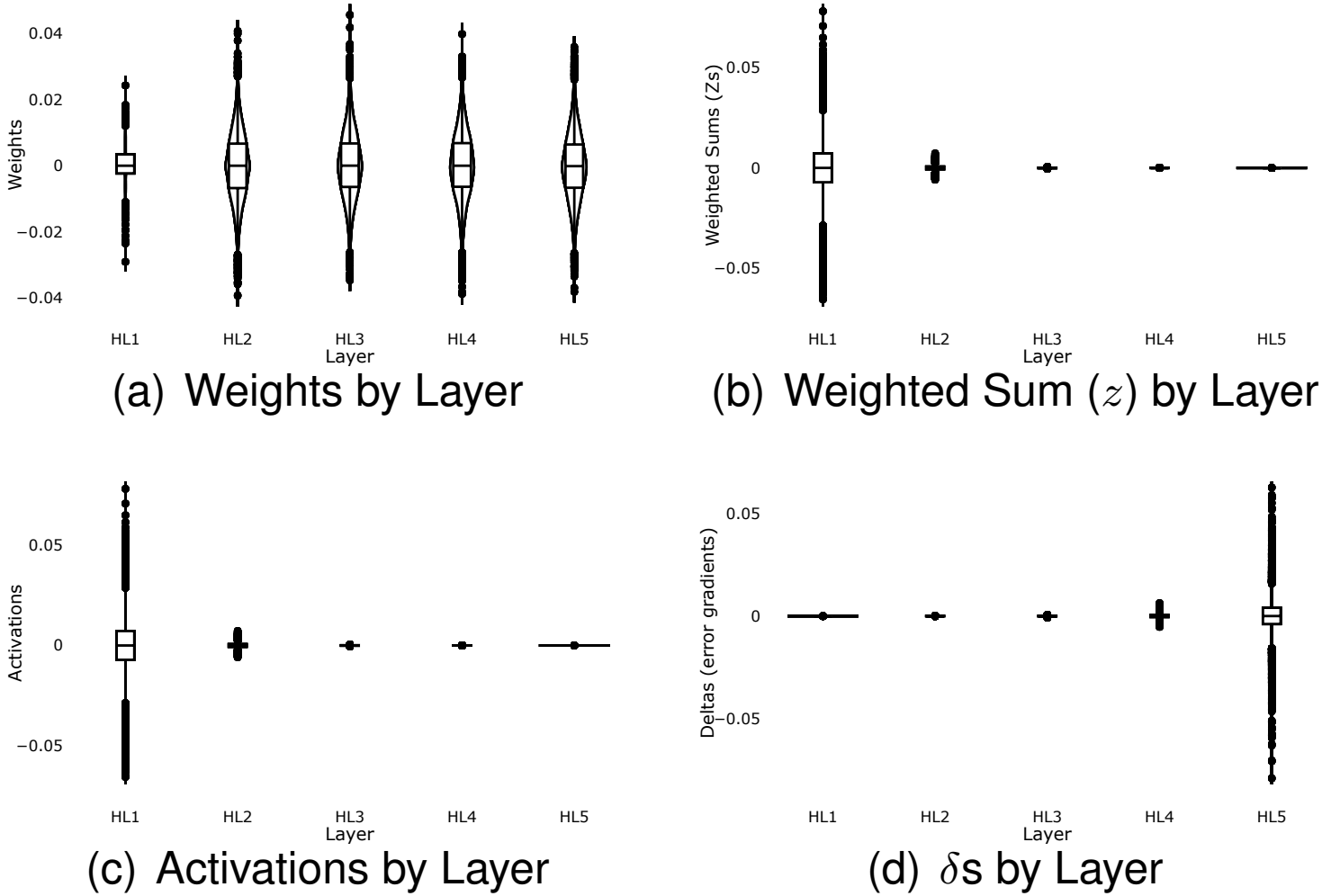


Figure 5: The architecture of the neural network used in the weight initialization experiments. Note that the neurons in this network use a linear activation function: $a_i = z_i$.

Weight Initialization and Unstable Gradients

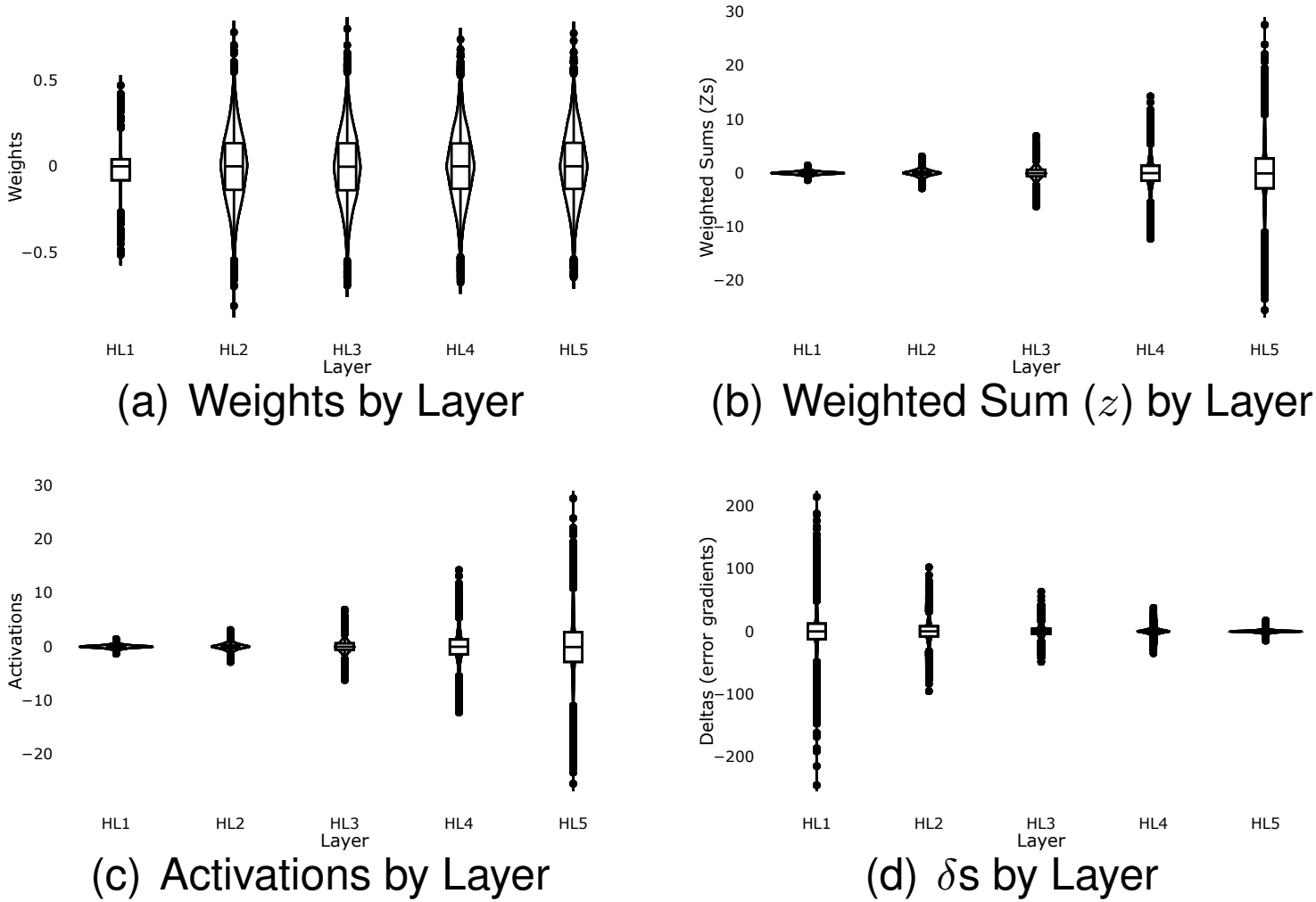


(a) Weights by Layer (b) Weighted Sum (z) by Layer

(c) Activations by Layer (d) δs by Layer

Figure 6: The internal dynamics of the network in Figure 5[19] during the first training iteration when the weights were initialized using a normal distribution with μ=0.0, σ=0.01.

Weight Initialization and Unstable Gradients



(a) Weights by Layer (b) Weighted Sum (z) by Layer

(c) Activations by Layer (d) δs by Layer

Figure 7: The internal dynamics of the network in Figure 5[19] during the first training iteration when the weights were initialized using a normal distribution with $\mu = 0.0$ and $\sigma = 0.2$.

$$z = (w_1 \times d_1) + (w_2 \times d_2) + \cdots + (w_{n_{in}} \times d_{n_{in}}) \tag{14}$$

$$var\left(\sum_{i=1}^{n} X_i\right) = \sum_{i=1}^{n} var\left(X_i\right) \tag{15}$$

$$
\begin{aligned}
var(z) &= var\left((w_1 \times d_1) + (w_2 \times d_2) + \ldots (w_{n_{in}} \times d_{n_{in}})\right) \\
&= \sum_{i=1}^{n_{in}} var(w_i \times d_i) \qquad (16)
\end{aligned}
$$

$$var(w \times d) = [E(\mathbf{W})]^2\, var(\mathbf{d}) + [E(\mathbf{d})]^2\, var(\mathbf{W}) + var(\mathbf{W})\, var(\mathbf{d}) \qquad (17$$

$$var(w \times d) = var(\mathbf{W}) var(\mathbf{d}) \qquad (18)$$

$$var(z) = \sum_{i=1}^{n_{in}} var(w_i \times d_i) = n_{in}\ var(\mathbf{W})\ var(\mathbf{d}) \tag{19}$$

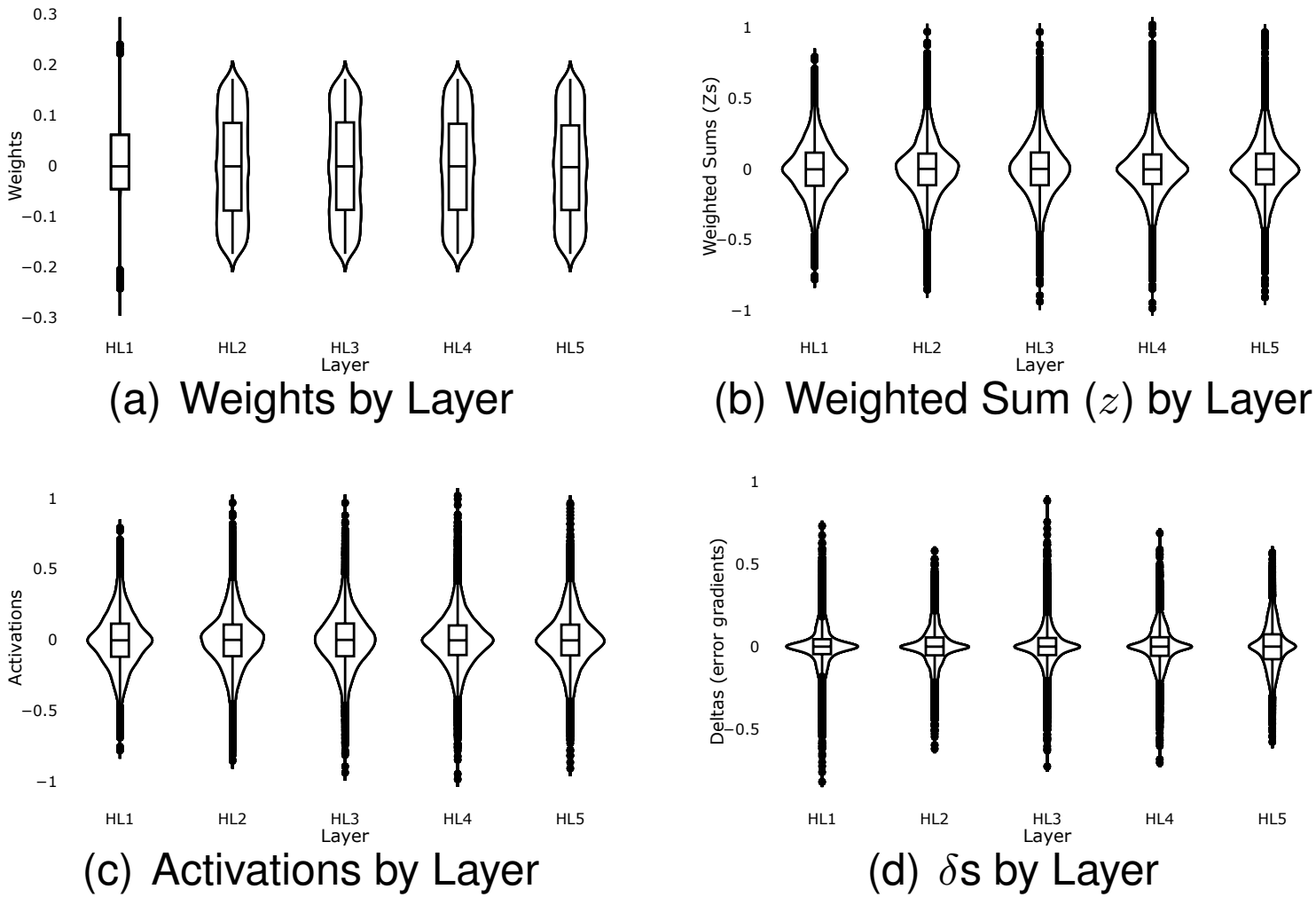
$$
\begin{aligned}
var(Z^{(HL1)}) &= n_{in}^{(HL1)} \times var(\mathbf{W}^{(HL1)}) \times var(\mathbf{d}^{(HL1)}) \qquad (20) \\
&= 2 \times 0.0001 \times 1 \\
&= 0.0002
\end{aligned}
$$

$$
\begin{aligned}
var(Z^{(HL2)}) &= n_{in}^{(HL2)} \times var(\mathbf{W}^{(HL2)}) \times var(\mathbf{d}^{(HL2)}) \qquad (21) \\
&= 100 \times 0.0001 \times 0.0002 \\
&= 0.000002
\end{aligned}
$$

$$var(\mathbf{W}^{(k)}) = \frac{2}{n_{in}^{(k)} + n_{out}^{(k)}} \tag{22}$$

$$var(\mathbf{W}^{(k)}) = \frac{1}{n_{in}^{(k)}} \tag{23}$$

Weight Initialization and Unstable Gradients



(a) Weights by Layer (b) Weighted Sum (z) by Layer

(c) Activations by Layer (d) δs by Layer

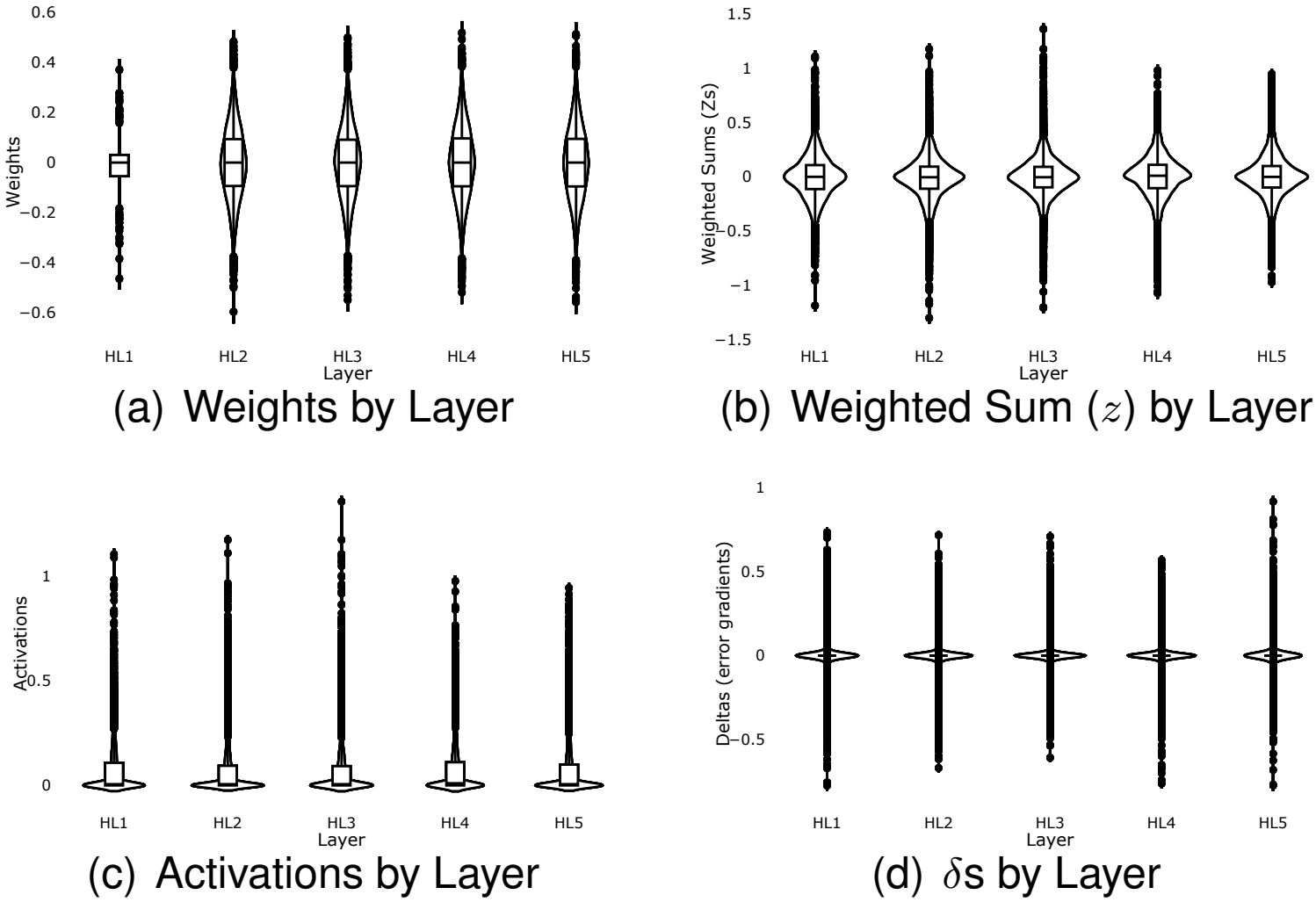
Figure 8: The internal dynamics of the network in Figure 5[19] during the first training iteration when the weights were initialized using Xavier initialization.

$$var(\mathbf{W}^{(k)}) = \frac{2}{n_{in}^{(k)}} \tag{24}$$

- Fully Connected Three-Layer ReLU Network:
  - Input Layer: 100 neurons
  - First Hidden Layer: 80 neurons (ReLU activation)
  - Second Hidden Layer: 50 neurons (ReLU activation)
  - Output Layer: 5 neurons

$$
\begin{aligned}
W^{(1)} &\sim \mathcal{N}\left(0, \sqrt{\frac{1}{100}}\right) \\
W^{(2)} &\sim \mathcal{N}\left(0, \sqrt{\frac{2}{80}}\right) \\
W^{(3)} &\sim \mathcal{N}\left(0, \sqrt{\frac{2}{50}}\right)
\end{aligned} \tag{25}
$$

Weight Initialization and Unstable Gradients



(a) Weights by Layer (b) Weighted Sum (z) by Layer

(c) Activations by Layer (d) δs by Layer

Figure 9: The internal dynamics of the network in Figure 5[19], using ReLUs, during the first training iteration when the weights were initialized using He initialization.

Handling Categorical Target Features: Softmax Output Layers and Cross-Entropy Loss Functions

1. represent the target feature using **one-hot encoding**;
2. change the output layer of the network to be a **softmax layer**; and
3. change the error (or loss) function we use for training to be the **cross-entropy** function.

Table 4: The *range-normalized* hourly samples of ambient factors and full load electrical power output of a combined cycle power plant, rounded to two decimal places, and with the (binned) target feature represented using one-hot encoding.

ID	AMBIENT TEMPERATURE	RELATIVE HUMIDITY	Electrical Output		
	°C	%	*low*	*medium*	*high*
1	0.04	0.81	0	0	1
2	0.84	0.58	1	0	0
3	0.50	0.07	0	1	0
4	0.53	1.00	0	1	0

Handling Categorical Target Features: Softmax Output Layers and Cross-Entropy Loss Functions

$$\varphi_{sm}\left(z_i\right) = \frac{e^{z_i}}{\sum_{j=1}^{m} e^{z_j}} \tag{26}$$

Table 5: The calculation of the softmax activation function φ_{sm} over a vector of three logits $\mathbf{l}$.

	$\mathbf{l}_0$	$\mathbf{l}_1$	$\mathbf{l}_2$
$\mathbf{l}$	1.5	-0.9	0.6
$e^{\mathbf{l}_i}$	4.48168907	0.40656966	1.8221188
$\sum_i e^{\mathbf{l}_i}$			6.71037753
$\varphi_{sm}(\mathbf{l}_i)$	0.667874356	0.060588195	0.27153745

Handling Categorical Target Features: Softmax Output Layers and Cross-Entropy Loss Functions

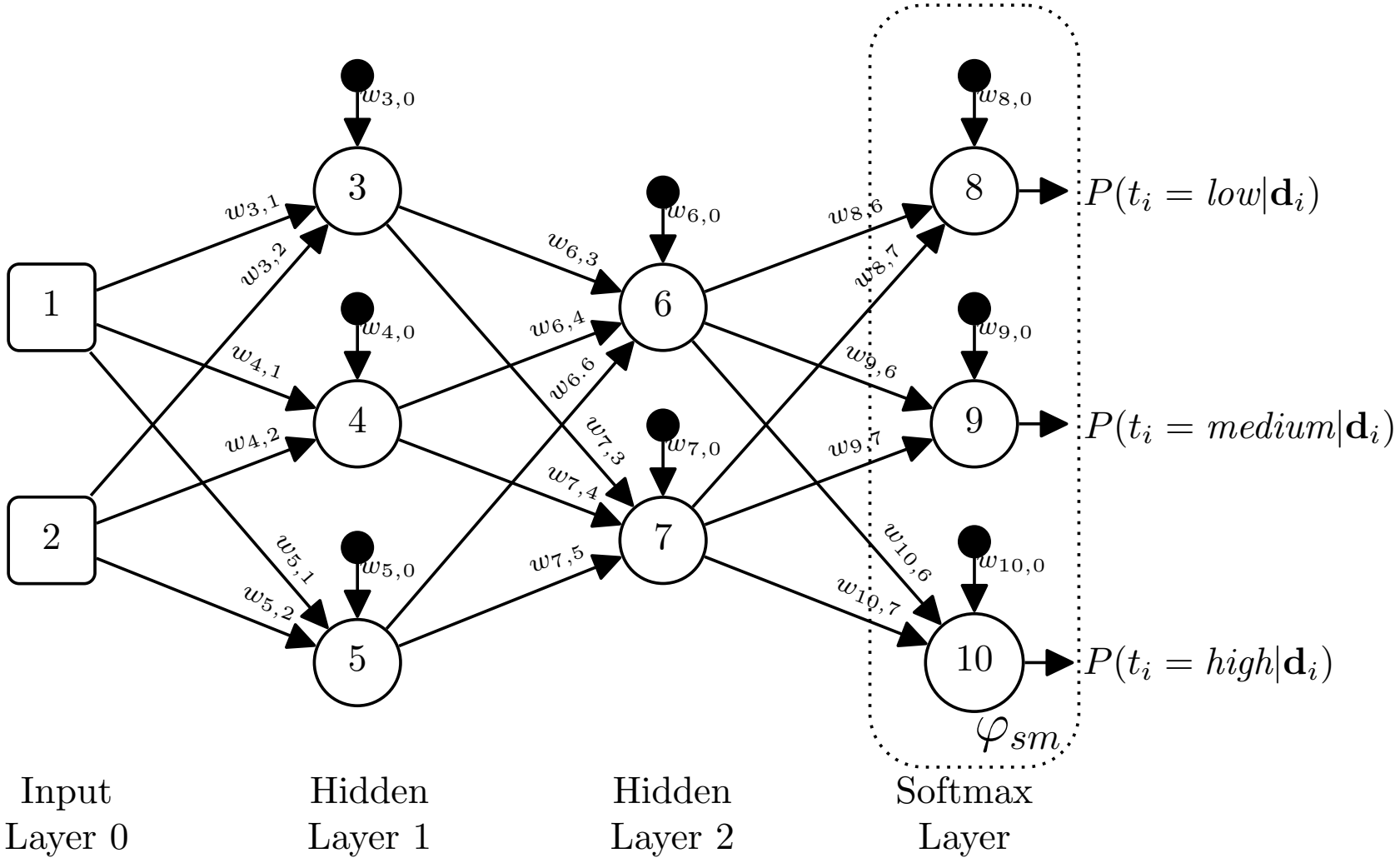


Figure 10: A schematic of a feedforward artificial neural network with a three-neuron softmax output layer.

Handling Categorical Target Features: Softmax Output Layers and Cross-Entropy Loss Functions

$$L_{CE}\left(\mathbf{t}, \hat{\mathbf{P}}\right) = -\sum_{j} \mathbf{t}_j \ln\left(\hat{\mathbf{P}}_j\right) \tag{27}$$

$$L_{CE}\left(\mathbf{t}, \hat{\mathbf{P}}\right) = -\ln\left(\hat{\mathbf{P}}_\star\right) \tag{28}$$

Handling Categorical Target Features: Softmax Output Layers and Cross-Entropy Loss Functions

$$
\begin{aligned}
L_{CE}\left(\mathbf{t}, \hat{\mathbf{P}}\right) &= -\sum_{j} \mathbf{t}_j \ln\left(\hat{\mathbf{P}}_j\right) \\
&= -\left(\left(\mathbf{t}_0 \ln\left(\hat{\mathbf{P}}_0\right)\right) + \left(\mathbf{t}_1 \ln\left(\hat{\mathbf{P}}_1\right)\right) + \left(\mathbf{t}_2 \ln\left(\hat{\mathbf{P}}_2\right)\right)\right) \\
&= -\left(\left(0 \ln\left(\hat{\mathbf{P}}_0\right)\right) + \left(1 \ln\left(\hat{\mathbf{P}}_1\right)\right) + \left(0 \ln\left(\hat{\mathbf{P}}_2\right)\right)\right) \\
&= -1 \ln\left(\hat{\mathbf{P}}_1\right) \qquad (29)
\end{aligned}
$$

Handling Categorical Target Features: Softmax Output Layers and Cross-Entropy Loss Functions

$$\boldsymbol{\delta}_k = \frac{\partial \mathcal{E}}{\partial z_k} \tag{30}$$

$$= \frac{\partial L_{CE}\left(\mathbf{t}, \hat{\mathbf{P}}\right)}{\partial \mathbf{l}_k} \tag{31}$$

$$= \frac{\partial - \ln\left(\hat{\mathbf{P}}_\star\right)}{\partial \mathbf{l}_k} \tag{32}$$

$$= \frac{\partial - \ln\left(\hat{\mathbf{P}}_\star\right)}{\partial\left(\hat{\mathbf{P}}_\star\right)} \times \frac{\partial\left(\hat{\mathbf{P}}_\star\right)}{\partial \mathbf{l}_k} \tag{33}$$

Handling Categorical Target Features: Softmax Output Layers and Cross-Entropy Loss Functions

$$\frac{d \ln x}{dx} = \frac{1}{x} \tag{34}$$

$$\frac{\partial - \ln\left(\hat{\mathbf{P}}_\star\right)}{\partial\left(\hat{\mathbf{P}}_\star\right)} = -\frac{1}{\hat{\mathbf{P}}_\star} \tag{35}$$

$$\frac{\partial\left(\hat{\mathbf{P}}_\star\right)}{\partial \mathbf{l}_k} = \begin{cases} \hat{\mathbf{P}}_\star\left(1 - \hat{\mathbf{P}}_k\right) & \text{if } k = \star \\ -\hat{\mathbf{P}}_\star \hat{\mathbf{P}}_k & otherwise \end{cases} \tag{36}$$

Handling Categorical Target Features: Softmax Output Layers and Cross-Entropy Loss Functions

$$\boldsymbol{\delta}_k = \frac{\partial - \ln\left(\hat{\mathbf{P}}_\star\right)}{\partial\left(\hat{\mathbf{P}}_\star\right)} \times \frac{\partial\left(\hat{\mathbf{P}}_\star\right)}{\partial \mathbf{l}_k} \tag{37}$$

$$= -\frac{1}{\hat{\mathbf{P}}_\star} \times \frac{\partial\left(\hat{\mathbf{P}}_\star\right)}{\partial \mathbf{l}_k} \tag{38}$$

$$= -\frac{1}{\hat{\mathbf{P}}_\star} \times \begin{cases} \hat{\mathbf{P}}_\star\left(1 - \hat{\mathbf{P}}_k\right) & \text{if } k = \star \\ -\hat{\mathbf{P}}_\star \hat{\mathbf{P}}_k & otherwise \end{cases} \tag{39}$$

$$= \begin{cases} -\left(1 - \hat{\mathbf{P}}_k\right) & \text{if } k = \star \\ \hat{\mathbf{P}}_k & otherwise \end{cases} \tag{40}$$

$$\boldsymbol{\delta}_{k=\star} = -\left(1 - \hat{\mathbf{P}}_k\right) \tag{41}$$

$$\boldsymbol{\delta}_{k\neq\star} = \hat{\mathbf{P}}_k \tag{42}$$

Handling Categorical Target Features: Softmax Output Layers and Cross-Entropy Loss Functions

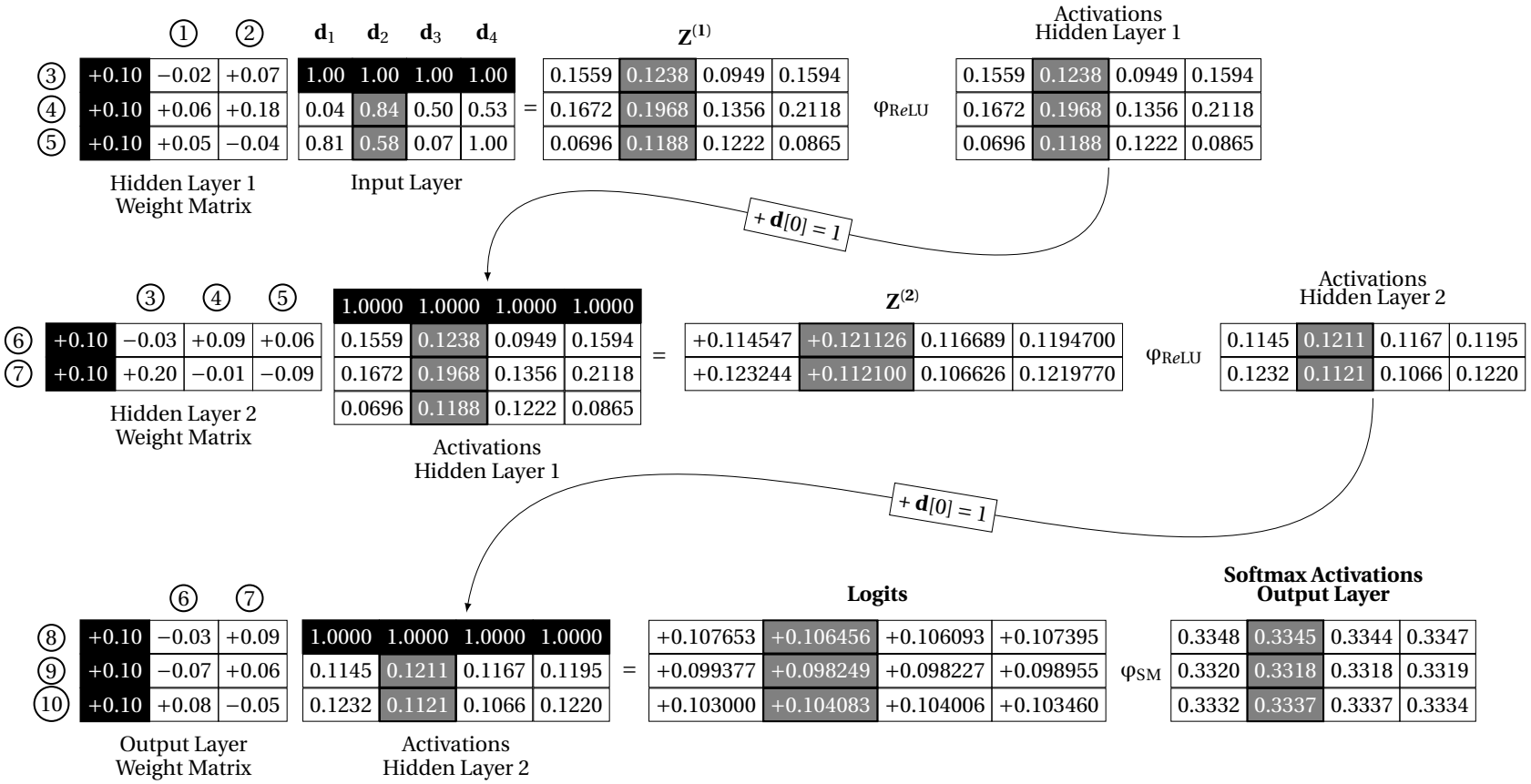


Figure 11: The forward pass of the mini-batch of examples listed in Table 4[34] through the network in Figure 10[37].

Table 6: The calculation of the softmax activations for each of the neurons in the output layer for each example in the mini-batch, and the calculation of the δ for each neuron in the output layer for each example in the mini-batch.

	$\mathbf{d}_1$	$\mathbf{d}_2$	$\mathbf{d}_3$	$\mathbf{d}_4$
Per Neuron Per Example logits				
Neuron 8	0.107653	0.106456	0.106093	0.107395
Neuron 9	0.099377	0.098249	0.098227	0.098955
Neuron 10	0.103000	0.104083	0.1040060	0.103460
Per Neuron Per Example $e^{\mathbf{l}_i}$				
Neuron 8	1.113661238	1.112328983	1.111925281	1.11337395
Neuron 9	1.104482611	1.103237457	1.103213186	1.104016618
Neuron 10	1.108491409	1.109692556	1.109607113	1.109001432
$\sum_i e^{\mathbf{l}_i}$	3.326635258	3.325258996	3.324745579	3.326392
Per Neuron Per Example Softmax Activations				
Neuron 8	0.3348	0.3345	0.3344	0.3347
Neuron 9	0.3320	0.3318	0.3318	0.3319
Neuron 10	0.3332	0.3337	0.3337	0.3334
Per Neuron Target One-Hot Encodings				
Neuron 8	0	1	0	0
Neuron 9	0	0	1	1
Neuron 10	1	0	0	0
Per Neuron Per Example δs				
Neuron 8	0.3348	-0.6655	0.3344	0.3347
Neuron 9	0.3320	0.3318	-0.6682	-0.6681
Neuron 10	-0.6668	0.3337	0.3337	0.3334

$$
\begin{aligned}
\Delta w_{9,6} &= \sum_{j=1}^{4} \boldsymbol{\delta}_{9,j} \times a_{6,j} \\
&= (0.3320 \times 0.1145) + (0.3318 \times 0.1211) \\
&\quad + (-0.6682 \times 0.1167) + (-0.6681 \times 0.1195) \\
&= 0.038014 + 0.04018098 + -0.07797894 + -0.07983795 \\
&= -0.07962191
\end{aligned} \tag{43}
$$

$$
\begin{aligned}
w_{9,6} &= w_{9,6} - \alpha \times \Delta w_{9,6} \\
&= -0.07 - 0.01 \times -0.07962191 \\
&= -0.07 - (-0.000796219) \\
&= -0.069203781
\end{aligned} \tag{44}
$$

Algorithm 1 The early stopping algorithm

```
Require: p the patience parameter
Require: D_ν a validation set
bestValidationError = ∞
tmpValidationError = 0
θ = initial model parameters
θ^best = 0
patienceCount = 0
while patienceCount < p do
    θ = new model parameters after most recent weight update
    tmpValidationError = calculateValidationError(θ, D_ν)
    if bestValidationError ≥ tmpValidationError then
        bestValidationError = tmpValidationError
        θ^best = θ
        patienceCount = 0
    else
        patienceCount = patienceCount + 1
    end if
end while
return Best Model Parameters θ^best
```

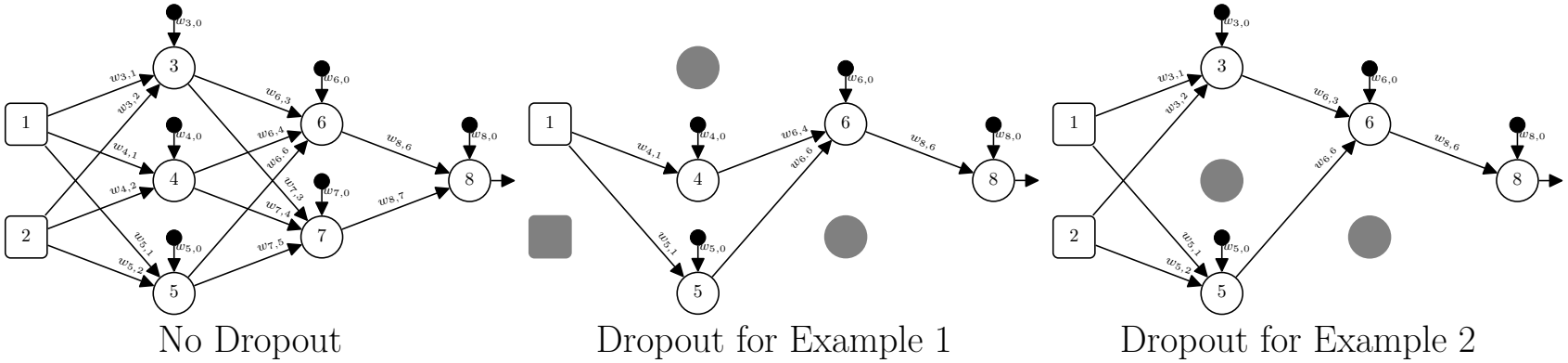


Figure 12: An illustration of how different small networks are generated for different training examples by applying dropout to the original large network. The gray nodes mark the neurons that have been dropped from the network for the training example.

Algorithm 2 Extensions to Backpropagation to Use Inverted Dropout

Require: ρ probability that a neuron in a layer will not be dropped

▷ Forward Pass

for each input or hidden layer l **do**

  $\mathbf{DropMask}^{(l)} = (m_1, \ldots, m_{size(l)}) \sim Bernoulli(\rho)$

  $\mathbf{a}^{(l)\prime} = \mathbf{a}^{(l)} \odot \mathbf{DropMax}^{(l)}$

  $\mathbf{a}^{(l)\prime\prime} = \frac{1}{\rho}\mathbf{a}^{(l)\prime}$

end for

▷ Backward Pass

for each layer l in backward pass **do**

  $\delta^{(l)} = \delta^{(l)} \odot \mathbf{DropMax}^{(l)}$

end for